PROJECT REPORT

Snake Game — Python

Member	Contribution
Abir (22-47645-2)	Starting part, Background , Snake Head
Tahmid (23-50426-1)	Snake Food, Snake Body, Border Collision
Mim (22-47409-2)	Body Collision, Scoring System

Course	Introduction to Python Programming
Language	Python
Library	turtle, time, random (standard library)
Date	**28** March 2026

1. Introduction

This report documents the development of a Snake Game built using Python's built-in **turtle** graphics library. The project was created as a beginner-level programming exercise by a team of three students — Abir, Tahmid, and Mim. The game replicates the classic Snake arcade experience: the player controls a snake that grows longer each time it eats food, while avoiding collisions with the walls and its own body.

The project was divided into three distinct sections so each team member could contribute independently, then integrate their work into one fully functional program.

2. Objectives

- Build a playable Snake game using only Python's standard library.
- Practise fundamental programming concepts: variables, loops, functions, and conditionals.
- Understand event-driven programming through keyboard input handling.
- Implement real-time collision detection for walls and the snake's own body.
- Track and display a live score and a persistent high score.
- Divide work fairly across three team members and integrate the parts seamlessly.

3. Tools and Technologies

Tool / Library	Purpose
Python	Core programming language
turtle	Graphics window, shapes, drawing, and keyboard events
time	Controlling game speed via time.sleep()
random	Randomly repositioning food after each eat

4. Team Contributions

4.1 Abir — Starting part, Background and Snake Head

Abir was responsible for setting up the entire project foundation and implementing the snake's head — the most critical moving element of the game.

- **Project setup:** Initialised global variables (delay, score, high_score), created the turtle Screen with a light-blue background (600x600 px), and disabled the automatic screen tracer for manual frame control.

- **Scoreboard pen:** Created a hidden turtle used exclusively to write the score text at the top of the screen.
- **Snake head:** Configured the head turtle as a black square at the centre of the screen with an initial direction of 'stop'.
- **Direction functions:** Wrote go_up(), go_down(), go_left(), and go_right() with reverse-direction guards to prevent 180-degree turns.
- **move() function:** Moves the head 20 pixels per tick in whichever direction is currently set.
- **Keybindings:** Mapped W / A / S / D to the four direction functions using wn.onkeypress().

4.2 Tahmid — Snake Food, Snake Body and Border Collision

Tahmid implemented the interactive elements that make the game challenging — the food, the growing body, and the wall boundaries.

- **Snake food:** Created a red circle turtle placed initially at (0, 100). When eaten, the food teleports to a random position within the play area.
- **Snake body segments:** Each time the snake eats food, a new grey square turtle is created and appended to the segments list.
- **Body movement:** Implemented a reverse-iteration loop so each segment follows the position of the segment ahead of it, giving a smooth chain effect.
- **Speed increase:** Reduces the delay variable by 0.001 seconds each time food is eaten, making the snake progressively faster.
- **Border collision:** Detects when the head crosses +-290 on either axis, resets the snake to the origin, clears all segments, and resets the score.

4.3 Mim — Body Collision and Scoring

Mim handled the game-over logic for self-collision and the complete scoring system.

- **Body collision detection:** After every move, iterates through all segments and checks if any segment is within 20 pixels of the head. If so, the game resets.
- **Score increment:** Adds 10 points each time the snake eats a piece of food.
- **High score tracking:** Compares the current score against the high score after every increment and updates it if a new record is reached.
- **Score display:** Clears and rewrites the scoreboard text after every change (food eaten, border hit, or body collision), keeping the display accurate at all times.

---

5. Code Explanation

The program is structured as a single-file script with a continuous game loop. Below is a walkthrough of the key sections.

5.1 Initialisation

All global variables and turtle objects are created before the loop begins. wn.tracer(0) disables automatic rendering so that wn.update() inside the loop controls exactly when each frame is drawn, preventing flickering.

5.2 Game Loop

The while True loop runs indefinitely. Each iteration: (1) updates the screen, (2) checks border collision, (3) checks food collision and grows the snake, (4) moves the body segments, (5) moves the head, (6) checks body collision, and (7) sleeps for the current delay value.

5.3 Collision Reset Logic

Both the border-collision and body-collision blocks share the same reset routine: sleep for 1 second so the player can see what happened, return the head to (0,0), send all body segments off-screen to (1000, 1000), clear the list, reset the score to 0, and redraw the scoreboard.

5.4 Key Code Snippet — Body Movement

```
for index in range(len(segments)-1, 0, -1):
    x = segments[index-1].xcor()
    y = segments[index-1].ycor()
    segments[index].goto(x, y)

if len(segments) > 0:
    x = head.xcor()
    y = head.ycor()
    segments[0].goto(x, y)
```

Iterating in reverse ensures that each segment receives the previous segment's position before that segment moves, so the chain effect is preserved correctly.

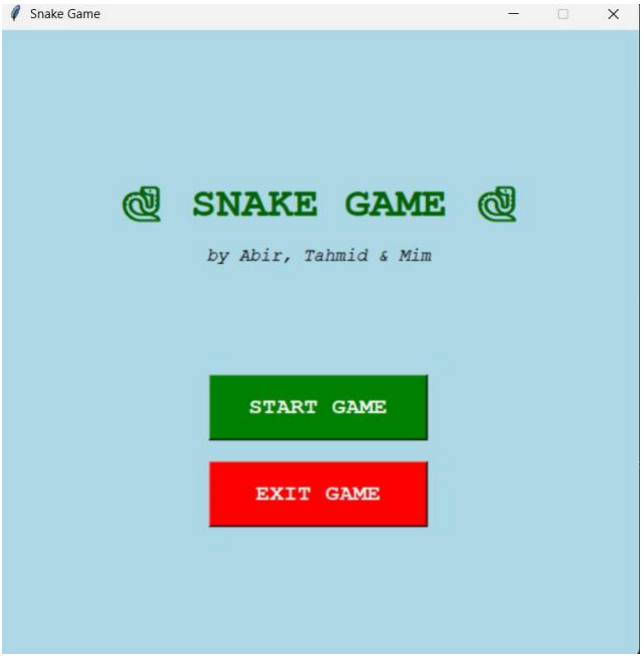
---

6. Features

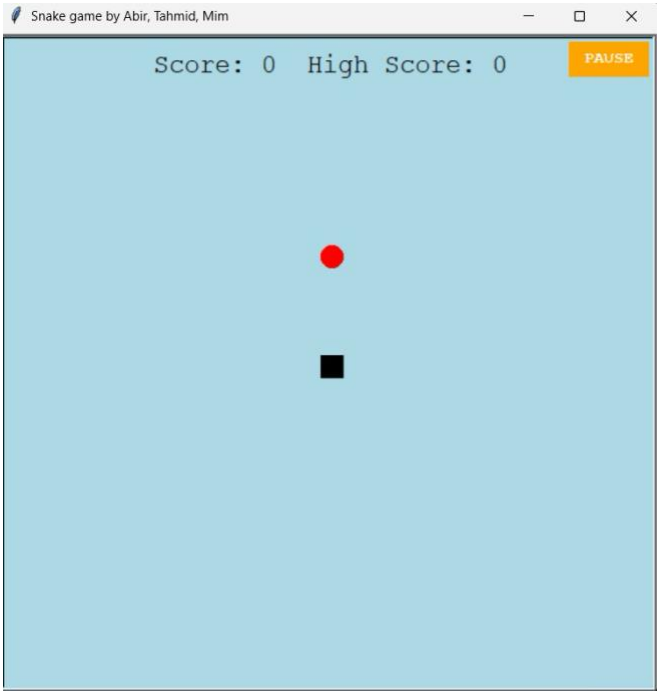
Feature	Description
Real-time movement	Snake moves continuously using W/A/S/D keys
Growing body	A new grey segment is added for every food eaten
Randomised food	Food respawns at a random position after being eaten
Progressive difficulty	Game speed increases slightly with every food eaten
Border collision	Game resets when the snake hits any wall

Self-collision	Game resets when the snake bites its own body
Live score	Current score shown at the top of the screen
High score	Best score preserved across resets within the session

Game menu:



Start game:



Pause game:

7. Challenges Faced

- **Reverse-direction prevention:** Without the direction guards in go_up() / go_down() etc., pressing the opposite key would instantly cause a self-collision. Solved by checking the current direction before updating it.
- **Body-movement ordering:** Moving segments from front to back instead of back to front caused every segment to jump to the head's position. Reversing the iteration order fixed this.
- **Frame control:** Turtle's default auto-update caused visual tearing. Using wn.tracer(0) combined with explicit wn.update() calls resolved this.
- **Collision radius tuning:** The 20-pixel threshold for both food and body-collision detection was chosen to match the 20x20 px square size of each turtle object, ensuring accurate hit detection.

---

8. Conclusion

The Snake Game project successfully demonstrates core Python programming skills in a fun and interactive format. Each team member — Abir, Tahmid, and Mim — contributed a well-defined section of the codebase, and their work integrated cleanly into a single, fully functional game.

The project covers a broad range of beginner Python concepts including variables, functions, loops, conditionals, lists, event handling, and basic object usage via the turtle library. It serves

as a strong foundation for more advanced game development topics such as object-oriented design, sprite-based graphics, and persistent high-score storage.

9. Reference

https://www.youtube.com/watch?v=BP7KMIbvtOo&list=PLlEgNdBJEO-n8k9SR49AshB9j7b5lw7hZ